UT3 Pruebas y Depuración en el desarrollo de software

Oscar Rodríguez y Andrei Moisei

DW1A

Introducción a las Pruebas de Software

Las Pruebas de Software son un proceso de ejecución de un programa con el objetivo de encontrar errores.

Su importancia esta en reducir costes de mantenimiento, mejorar la seguridad y la experiencia del usuario.

Las pruebas son el mecanismo para verificar y validar la calidad. Sin pruebas, no hay garantía de calidad.

Pruebas en el proceso de desarrollo

Planificación en el Ciclo de Vida

- La planificación de pruebas define con antelación los objetivos, el alcance, los recursos, las herramientas y las responsabilidades, estableciendo criterios claros para evitar improvisaciones y reducir riesgos durante el desarrollo.
- Las pruebas se realizan a lo largo de todo el ciclo de vida del software, validando requisitos y diseños en fases tempranas y comprobando la implementación y el mantenimiento para detectar errores cuanto antes.

Tipos de Pruebas (Funcionales y Estructurales)

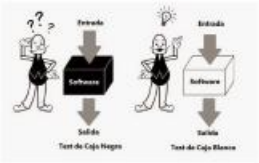
Funcionales
Verifican qué hace el sistema (cumplimiento de requisitos). Se centran en la salida correcta ante una entrada dada.

Estructurales
Verifican cómo está hecho el sistema (arquitectura, eficiencia, rutas de código).

Pruebas de Caja Blanca vs Caja Negra

•**Caja Negra (Black Box):**
No se mira el código fuente. Se prueba la interfaz y la funcionalidad.

•**Caja Blanca (White Box):**
Se analiza el código interno, bucles y condiciones lógica.



Pruebas de Regresión

- Las pruebas de regresión se ejecutan cada vez que se modifica el código para comprobar que los cambios no afectan negativamente a funcionalidades que ya funcionaban correctamente. Son especialmente importantes en proyectos grandes o con actualizaciones frecuentes.

Procedimientos de Prueba

Un procedimiento de prueba describe de forma detallada y ordenada los pasos que se deben seguir para ejecutar una prueba correctamente. Indica el entorno necesario, las acciones a realizar y el orden en el que deben ejecutarse, asegurando que la prueba pueda repetirse siempre de la misma manera.

El Caso de Prueba (Test Case)

•Un caso de prueba define qué se va a comprobar en una prueba concreta.
•Su objetivo es comparar ambos resultados para determinar si la prueba es correcta o no.
•**Elementos clave:**
1. **Entrada:** Datos que introducimos.
2. **Salida Esperada:** Lo que debería pasar.
3. **Resultado Real:** Lo que realmente pasó.

Planificación en el Ciclo de Vida

- La planificación de pruebas define con antelación los objetivos, el alcance, los recursos, las herramientas y las responsabilidades, estableciendo criterios claros para evitar improvisaciones y reducir riesgos durante el desarrollo.
- Las pruebas se realizan a lo largo de todo el ciclo de vida del software, validando requisitos y diseños en fases tempranas y comprobando la implementación y el mantenimiento para detectar errores cuanto antes.

Tipos de Pruebas (Funcionales y Estructurales)

Funcionales

Verifican qué hace el sistema (cumplimiento de requisitos). Se centran en la salida correcta ante una entrada dada.

Estructurales

Verifican cómo está hecho el sistema (arquitectura, eficiencia, rutas de código).

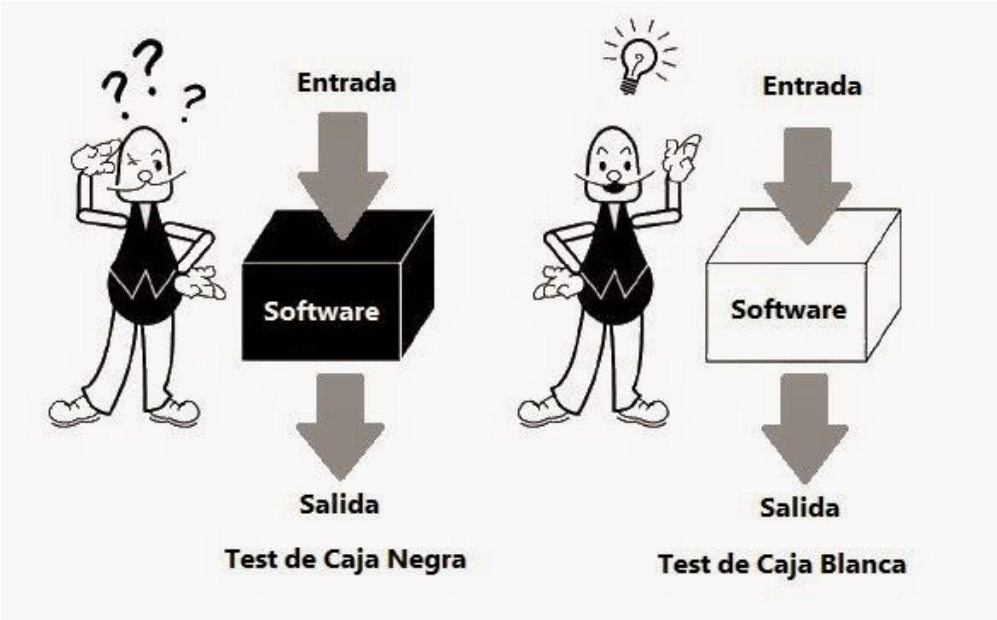
Pruebas de Caja Blanca vs Caja Negra

- **Caja Negra (Black Box):**

No se mira el código fuente. Se prueba la interfaz y la funcionalidad.

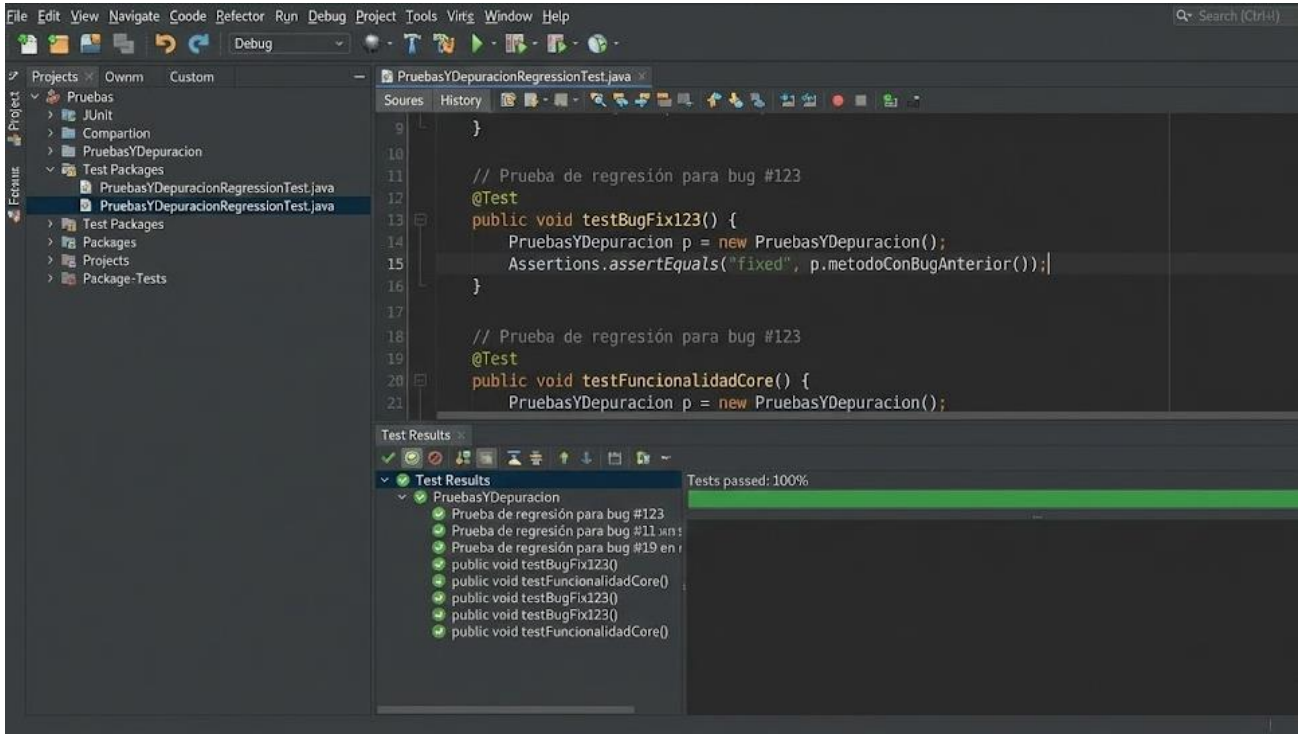
- **Caja Blanca (White Box):**

Se analiza el código interno, bucles y condiciones lógica.



Pruebas de Regresión

- Las pruebas de regresión se ejecutan cada vez que se modifica el código para comprobar que los cambios no afectan negativamente a funcionalidades que ya funcionaban correctamente. Son especialmente importantes en proyectos grandes o con actualizaciones frecuentes.



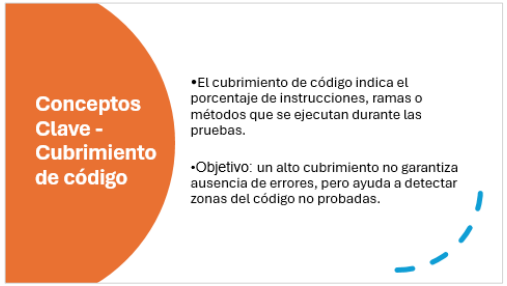
Procedimientos de Prueba

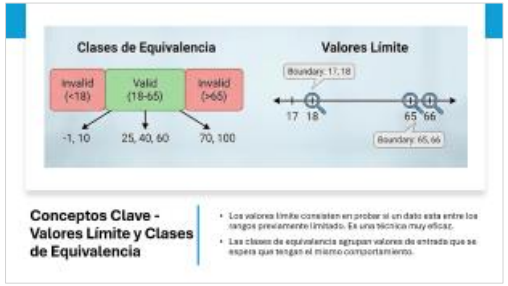
Un procedimiento de prueba describe de forma detallada y ordenada los pasos que se deben seguir para ejecutar una prueba correctamente. Indica el entorno necesario, las acciones a realizar y el orden en el que deben ejecutarse, asegurando que la prueba pueda repetirse siempre de la misma manera.

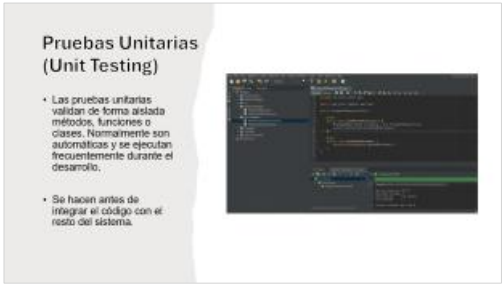
El Caso de Prueba (Test Case)

- Un caso de prueba define qué se va a comprobar en una prueba concreta.
- Su objetivo es comparar ambos resultados para determinar si la prueba es correcta o no.
- **Elementos clave:**
  1. **Entrada:** Datos que introducimos.
  2. **Salida Esperada:** Lo que debería pasar.
  3. **Resultado Real:** Lo que realmente pasó.

Pruebas de Código



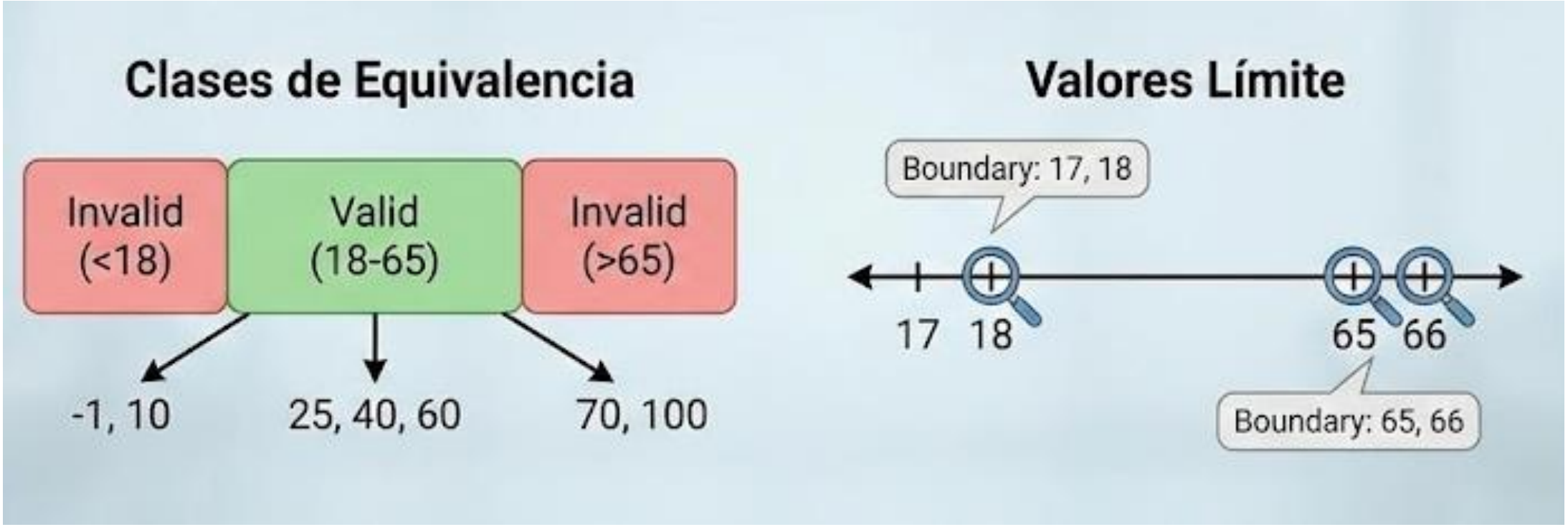




Conceptos Clave - Cubrimiento de código

- El cubrimiento de código indica el porcentaje de instrucciones, ramas o métodos que se ejecutan durante las pruebas.

- Objetivo: un alto cubrimiento no garantiza ausencia de errores, pero ayuda a detectar zonas del código no probadas.



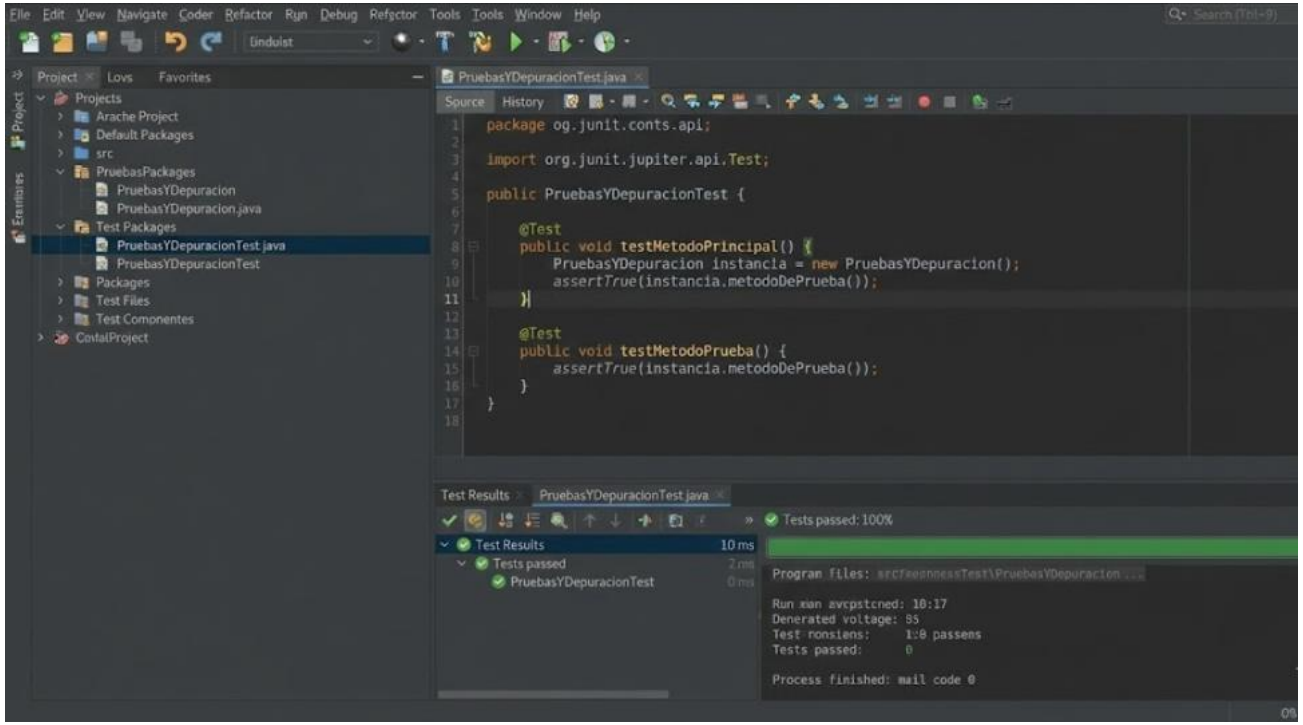
Conceptos Clave - Valores Límite y Clases de Equivalencia

- Los valores límite consisten en probar si un dato esta entre los rangos previamente limitado. Es una técnica muy eficaz.
- Las clases de equivalencia agrupan valores de entrada que se espera que tengan el mismo comportamiento.

Pruebas Unitarias (Unit Testing)

- Las pruebas unitarias validan de forma aislada métodos, funciones o clases. Normalmente son automáticas y se ejecutan frecuentemente durante el desarrollo.

- Se hacen antes de integrar el código con el resto del sistema.



Ventajas de las Pruebas Unitarias

- Detección temprana de errores (más barato de arreglar).
- Facilitan la refactorización (cambiar código sin miedo a romperlo).
- Sirven como documentación técnica del código.

Herramientas de Testing (xUnit)

Dependiendo del lenguaje se utilizan distintas herramientas:

- **Java:** JUnit, TestNG.
- **C#:** NUnit.
- **Python:** Pytest, unittest.
- **JavaScript:** Jest, Mocha.

Automatización de Pruebas

- La automatización de pruebas utiliza software para ejecutar pruebas y comparar resultados de manera rápida y confiable.
  - Pruebas manuales: Lentas, propensas a errores humanos y difíciles de repetir.
  - Pruebas automatizadas: Rápidas, repetibles indefinidamente y especialmente útiles para ejecutar pruebas de forma continua durante el desarrollo del software.

Pruebas de Integración

Las pruebas de integración comprueban que los distintos módulos o componentes del sistema funcionan correctamente al interactuar entre sí.

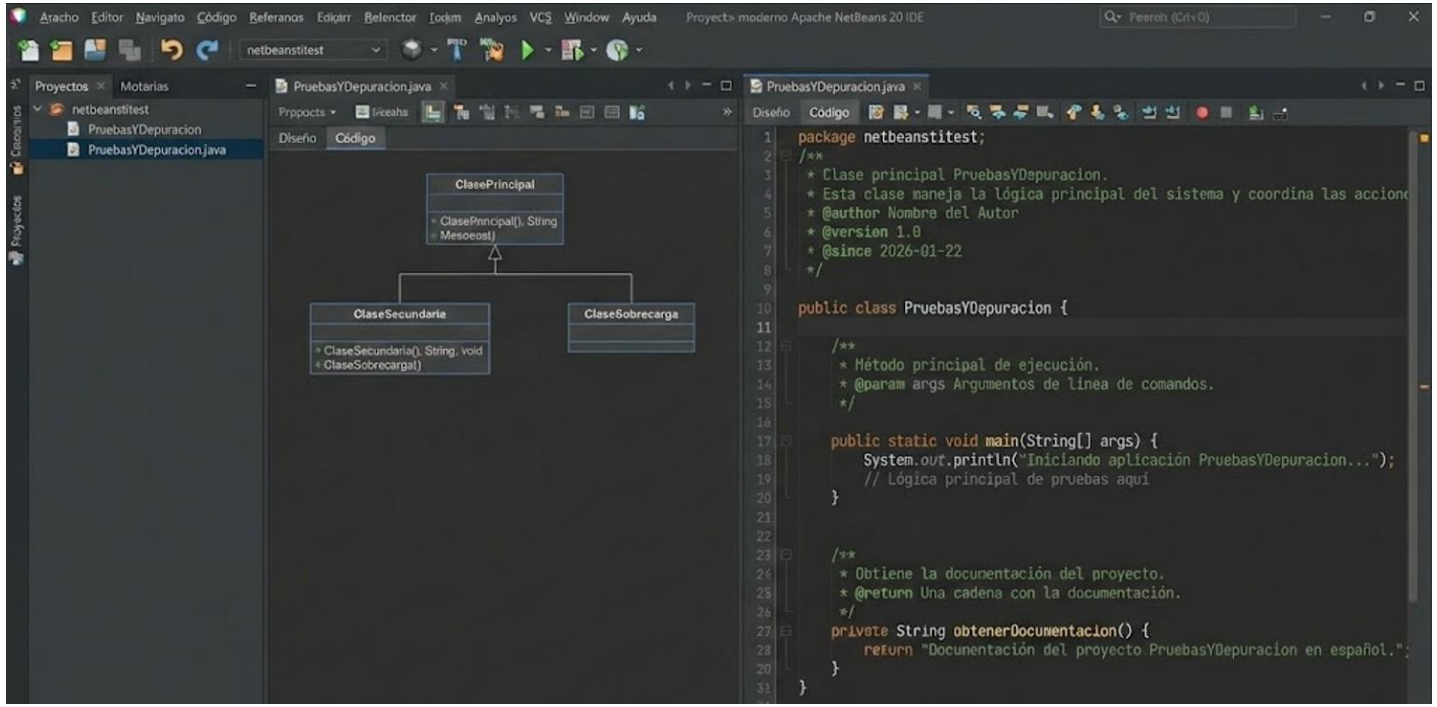
Diferencia: La prueba unitaria evalúa componentes individuales del sistema, mientras que la prueba de integración verifica la correcta interacción entre esos componentes

Se aplican después de las unitarias y antes de las de sistema.

Diseño y Documentación

El diseño de casos de prueba consiste en definir de forma clara y estructurada qué se va a probar, con qué datos y qué resultados se esperan obtener.

Una correcta documentación de los casos de prueba es fundamental para facilitar su comprensión, repetición y mantenimiento.

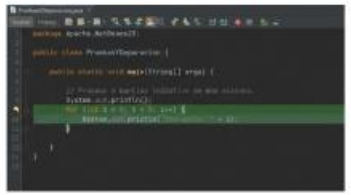


Depuración de Programas

¿Qué es la Depuración (Debugging)?

•La depuración es el proceso de localizar, analizar y corregir errores en un programa. Es una tarea esencial para garantizar el correcto funcionamiento del software.

•El testing encuentra el fallo y el debugging localiza la causa y la arregla.



Herramientas del IDE y Breakpoints

•Los entornos de desarrollo integrados (IDE) incorporan herramientas de depuración que permiten analizar el comportamiento del programa durante su ejecución.

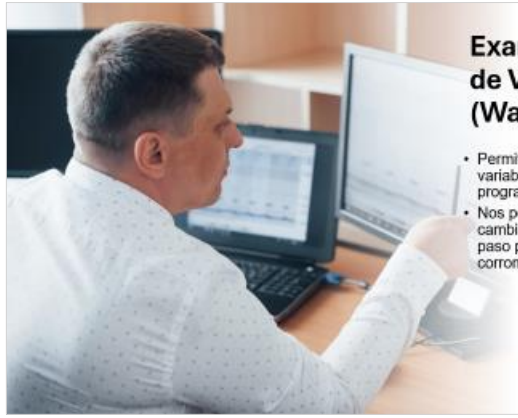
•Los *breakpoints* son puntos de parada que permiten detener la ejecución del programa en una línea concreta del código para analizar su comportamiento y el valor de las variables, facilitando la detección de errores.

Seguimiento en Tiempo de Ejecución

•**Step Over** (Pasar por encima): Ejecuta la línea actual sin entrar en detalles de funciones.

•**Step Into** (Entrar): Se mete dentro de la función llamada para verla línea a línea.

•**Step Out** (Salir): Termina la función actual y vuelve a quien la llamó.



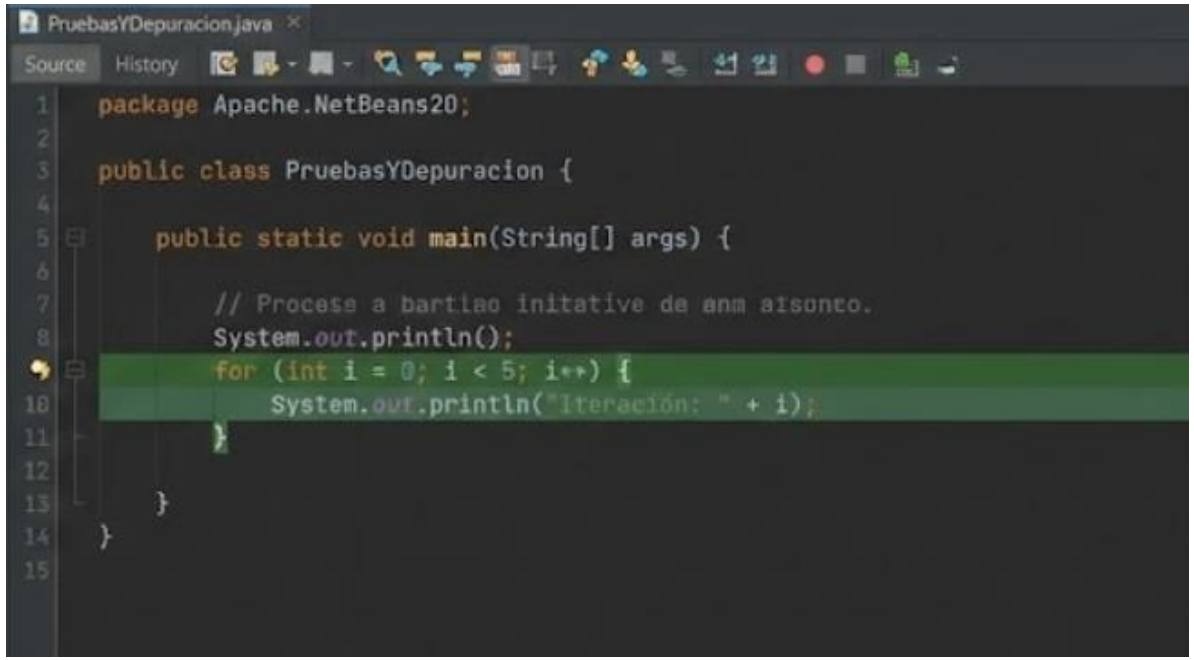
Examinadores de Variables (Watchers)

- Permiten ver el valor de una variable en vivo mientras el programa está detenido.
- Nos permite ver cómo cambian los datos paso a paso para detectar dónde se corrompe la información.

¿Qué es la Depuración (Debugging)?

•La depuración es el proceso de localizar, analizar y corregir errores en un programa. Es una tarea esencial para garantizar el correcto funcionamiento del software.

•El testing encuentra el fallo y el debugging localiza la causa y la arregla.



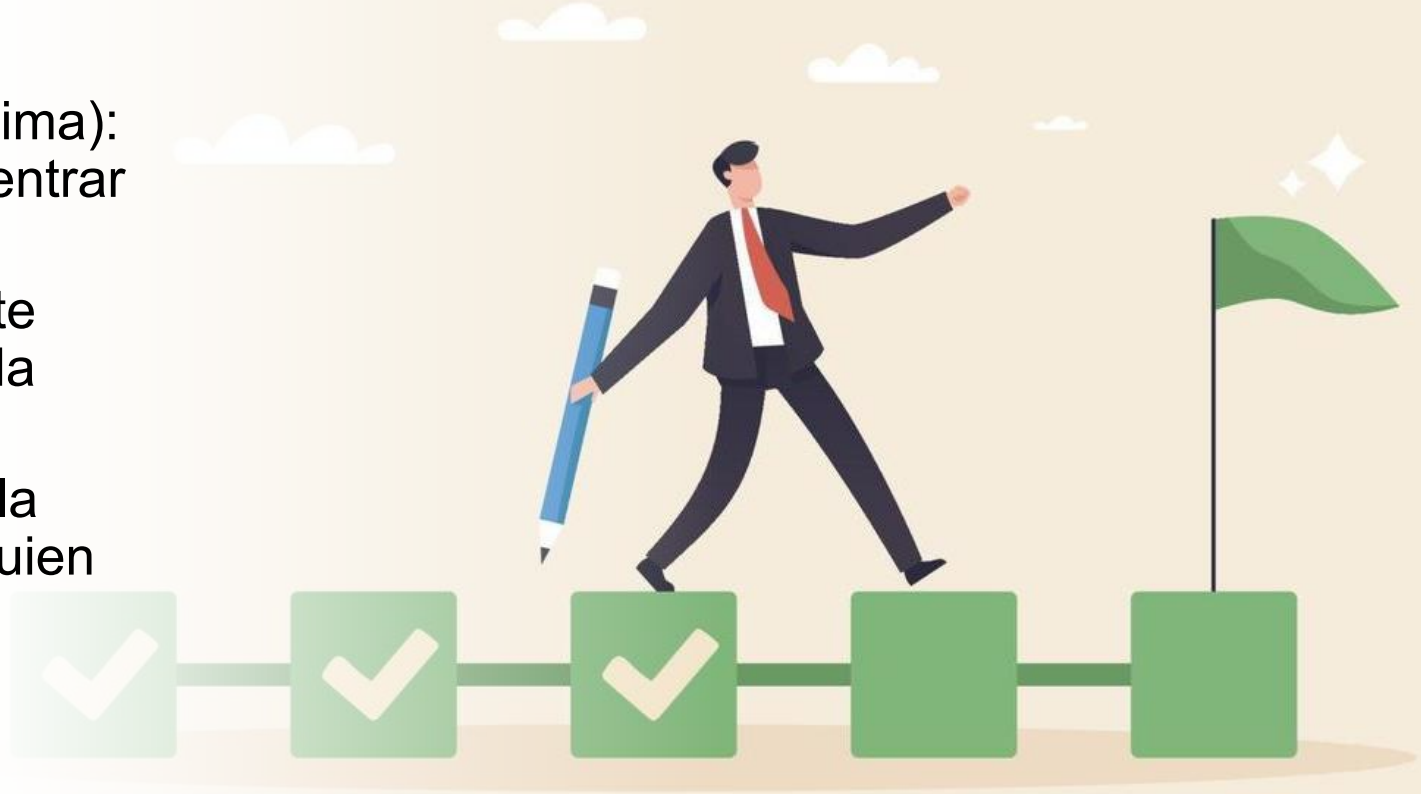
Herramientas del IDE y Breakpoints

- Los entornos de desarrollo integrados (IDE) incorporan herramientas de depuración que permiten analizar el comportamiento del programa durante su ejecución.

- Los *breakpoints* son puntos de parada que permiten detener la ejecución del programa en una línea concreta del código para analizar su comportamiento y el valor de las variables, facilitando la detección de errores.

Seguimiento en Tiempo de Ejecución

- **Step Over** (Pasar por encima): Ejecuta la línea actual sin entrar en detalles de funciones.
- **Step Into** (Entrar): Se mete dentro de la función llamada para verla línea a línea.
- **Step Out** (Salir): Termina la función actual y vuelve a quien la llamó.



Examinadores de Variables (Watchers)

- Permiten ver el valor de una variable en vivo mientras el programa está detenido.
- Nos permite ver cómo cambian los datos paso a paso para detectar dónde se corrompe la información.

Conclusión

- Las pruebas y la depuración son esenciales para garantizar la calidad del software, ya que permiten detectar y corregir errores de forma temprana.
- Su correcta aplicación mejora la fiabilidad, el mantenimiento y la seguridad de las aplicaciones, reduciendo fallos y costes a largo plazo.

Preguntas tipo test para examen

1. **¿Qué es una prueba unitaria?**

a) Una prueba del sistema completo
b) Una prueba de una clase o función individual ✅
c) Una prueba de aceptación
d) Una prueba manual

2. ¿Qué indica una alta cobertura de código?

a) Que no hay errores
b) Que la mayoría del código se ejecuta en las pruebas ✅
c) Que el código es más rápido
d) Que no se necesitan más pruebas

3. ¿Qué es un breakpoint?

a) Un error de compilación
b) Un punto donde se detiene la ejecución del programa ✅
c) Una prueba automática
d) Una clase especial

4. ¿Qué tipo de prueba verifica cambios recientes?

a) Funcional
b) Unitaria
c) Regresión ✅
d) Integración

5. ¿Qué herramienta se usa para pruebas en Java?

a) pytest
b) NUnit
c) JUnit ✅
d) Selenium